

JobReady AI

PHASE 2 | Technical Design Document

Turning a life-ending moment into a life-changing phase — through right guidance, awareness and information.

Document	Technical Design Document (TDD)
Product	JobReady AI
Version	v1.0 — Phase 2
Owner	Aditya Nagaria — Product & Program Manager
Purpose	Defines how JobReady AI is built — architecture, tech stack, data flow, prompt engine, LLM selection
Feeds into	Phase 3 — Build Sprint 1 (Core Engine)
Status	Draft — Phase 2 In Progress

1. PURPOSE OF THIS DOCUMENT

Phase 1 answered: *what are we building and why*. This document answers: *how are we going to build it*. It is the bridge between product discovery and the build sprint. It defines the system architecture, the technology choices and the reasons behind them, the data flow through each feature, the prompt engineering design, and the decision between Gemini and OpenAI as the LLM engine. Every decision here traces back to a requirement in the PRD and a moment in Rahul's journey.

2. SYSTEM ARCHITECTURE

The architecture is organised into five layers: User Input, Feature Modules, Prompt Engine, LLM Engine, and Output Layer. The design is intentionally LLM-agnostic — Gemini and OpenAI slot into the same layer and are interchangeable via a single config variable. The Application Tracker bypasses the LLM entirely in MVP — it is a data capture and display tool with no generative component. Google Colab is the runtime environment. GitHub stores all code and documentation.

USER INPUT

User profile Skills, experience, target role	Job description Pasted or uploaded JD text	Interview answer Typed user response
--	--	--



FEATURE MODULES

JD Analyser Fit score + gap analysis	Application Tracker Pipeline + dashboard	Mock Interview Engine Q gen + evaluation
--	--	--

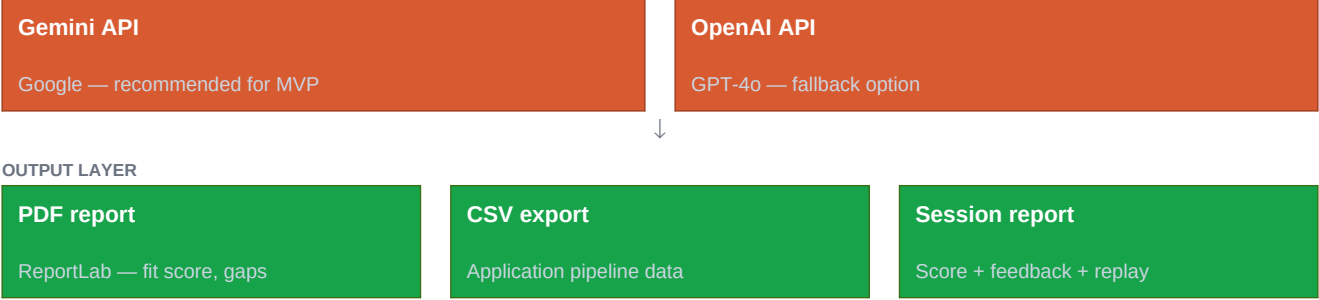
↓ (JD Analyser + Mock Engine only — Tracker bypasses LLM)

PROMPT ENGINE

Prompt builder Assembles context + template	Response parser Extracts + validates output
---	---



LLM ENGINE (INTERCHANGEABLE)



3. TECH STACK — DECISIONS & RATIONALE

TECHNOLOGY	ROLE	RATIONALE
Google Colab	Coding & runtime environment	Free, browser-based, no local setup. Shareable notebook = live demo + portfolio artifact in one. Native Google ecosystem alignment with Gemini API.
Python	Primary language	Universal for AI/ML tooling. All required libraries (google-generativeai, openai, reportlab, pandas) are Python-native. No context switching.
Gemini API	LLM engine (primary)	Free tier with generous limits. 1M token context window handles long JDs + full profile in one call. Native Colab fit. Already accessible. See Section 4 for full comparison.
OpenAI API	LLM engine (fallback)	GPT-4o mini as backup if Gemini evaluation depth falls short on Mock Interview Engine scoring. Architecture is LLM-agnostic — swap is one config line.
ReportLab	PDF generation	Pure Python PDF library. No external dependencies. Handles formatted reports, tables, and styled output. Used for JD Analyser PDF output and all Phase 1 documents.
Pandas	Data handling	Application Tracker data stored as a DataFrame in session. CSV export via df.to_csv(). Lightweight, no database required for MVP.
GitHub	Code storage & portfolio	Public repository for all code, notebooks, and documentation. Demonstrates both PM methodology and technical execution to interviewers and collaborators.
Hugging Face	Backup model option	Available if API costs become a constraint post-MVP. Open-source models via HF Inference API. Not in MVP scope.

4. LLM COMPARISON — GEMINI vs OPENAI

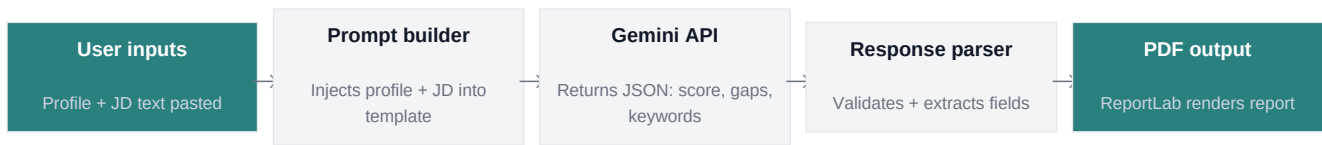
The architecture is LLM-agnostic by design. Both Gemini and OpenAI are evaluated against the same criteria relevant to JobReady AI's MVP. The decision is not permanent — it is the right choice for now, with a clear trigger condition for switching.

CRITERION	GEMINI (FLASH)	OPENAI (GPT-4o MINI)	WINNER
Free tier	Yes — generous limits	No — paid credits only	Gemini
Context window	1M tokens	128K tokens	Gemini
Cost per 1M tokens	~\$0.075 (input)	~\$0.15 (input)	Gemini
Colab integration	Native Google ecosystem	Works via openai SDK	Gemini
Setup ease	Google AI Studio — instant	Requires payment setup	Gemini
JSON output	Structured mode supported	Structured outputs supported	Tie
Gap analysis depth	Strong — good for JD Analyser	Strong — good for JD Analyser	Tie
Answer evaluation	Good	Slightly stronger instruction	OpenAI
Access available	Yes — already have access	Needs new API key + billing	Gemini

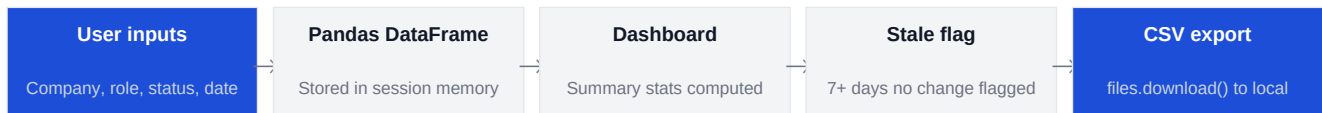
Decision: Gemini Flash for MVP. Start with Gemini — free, fast, native Colab fit, already accessible. Build LLM-agnostic (one config variable). Switch to OpenAI GPT-4o mini only if Mock Interview Engine answer evaluation quality is insufficient after 10+ prompt iterations.

5. DATA FLOW — FEATURE BY FEATURE

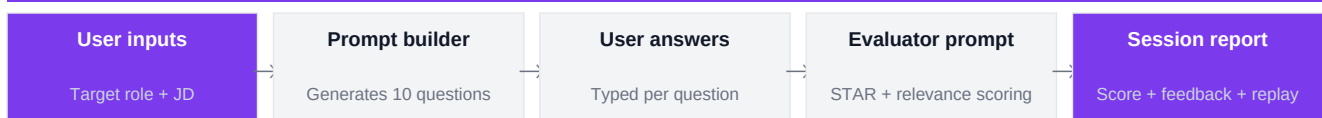
JD Analyser — data flow



Application Tracker — data flow



Mock Interview Engine — data flow



6. PROMPT ENGINE DESIGN

The prompt engine has two components: the Prompt Builder (what goes in) and the Response Parser (what comes out). It is shared across all LLM-dependent features. The engine is designed so that swapping Gemini for OpenAI requires changing only the API call — the prompt templates and parser logic remain identical.

6.1 Prompt Builder — three-layer structure

Layer 1: System prompt

Sets the LLM's role, persona, tone and output contract. Shared across all features with minor variation.

```
You are a career advisor for a mid-career professional who has been laid off after 11 years. Be specific, honest and warm. Never generic. Return ONLY valid JSON. No preamble. No explanation outside the JSON.
```

Layer 2: User context

Injects Rahul's profile, the JD text, experience, and target role. Makes every response personalised.

```
User profile: {name}, {years_exp} years experience, last role: {last_role}, target: {target_role}.  
Skills: {skills_list}. Job Description: {jd_text}
```

Layer 3: Task instruction

The feature-specific ask. Defines exactly what the LLM must return and in what structure.

```
Analyse the fit between this profile and JD. Return JSON with keys: fit_score (0-100), matched_skills (list), missing_skills (list), ats_keywords (list), recommendations (list of 3-5 strings).
```

6.2 Response Parser — three jobs

Extract

Parse the raw LLM string response and load as a Python dict using `json.loads()`. Strip any markdown fences (````json`) if present before parsing.

Validate

Check that all expected keys are present. Verify data types — `fit_score` must be an integer 0-100, lists must be lists. Log any missing keys.

Fallback

If parsing fails or required keys are absent, return a safe error dict with a user-friendly message and a retry flag. Never surface a Python traceback to the user.

7. LLM-AGNOSTIC DESIGN PATTERN

The architecture is built so that switching from Gemini to OpenAI (or any future LLM) requires changing exactly one configuration variable. The prompt templates, parser logic, and feature code are identical for both providers. This is implemented via a single `llm_call()` wrapper function.

```
# config.py – one line to switch LLM
LLM_PROVIDER = "gemini" # or "openai"

# llm_engine.py – agnostic wrapper
def llm_call(prompt: str) -> str:
    if LLM_PROVIDER == "gemini":
        model = genai.GenerativeModel('gemini-1.5-flash')
        return model.generate_content(prompt).text
    elif LLM_PROVIDER == "openai":
        response = openai.chat.completions.create(
            model='gpt-4o-mini',
            messages=[{'role': 'user', 'content': prompt}]
        )
        return response.choices[0].message.content
```

8. TECHNICAL ASSUMPTIONS, CONSTRAINTS & RISKS

ASSUMPTIONS	CONSTRAINTS
• Gemini API free tier remains available through the MVP build window • Google Colab session memory is sufficient for 50+ application records • LLM output will be JSON-parseable after prompt engineering iteration • User will self-report profile data accurately • English-language JDs are the primary input for MVP	• No persistent backend — all data is session-based in Colab MVP • LLM output is non-deterministic — same prompt may vary slightly • No real-time job market data — analysis is JD-text-based only • Token limits — JDs over ~8,000 words may need truncation • Solo builder — scope is constrained to 2-month bench window

RISK	LEVEL	SCENARIO	MITIGATION
LLM output quality	HIGH	Gemini returns inconsistent or unparseable JSON	Enforce JSON mode + strict prompt. Build robust fallback in parser. Test with 20+ JDs before release.
Session data loss	MED	User closes Colab without exporting CSV	Prompt user to export at session end. Show unsaved data warning. Future: Google Sheets integration.
Token overflow	MED	Long JD + full profile exceeds context limit	Truncate JD to first 3,000 words. Summarise profile to key fields only. Flash 1M window makes this low risk.
Prompt drift	LOW	LLM interprets prompt differently across versions	Pin model version in config. Log prompt + response pairs during testing. Version-control all prompts.

This document is the engineering foundation for Phase 3.

Every architecture decision here traces back to a user requirement in the PRD
and a moment in Rahul's journey. That is what makes it a product document, not just a technical
one.